

Innovatech Solutions

Descripción de la Persistencia de Datos

DSY1106 · Desarrollo Fullstack III · Evaluación Parcial N°3

Tecnología ORM	Prisma v5 (TypeScript/JavaScript)
Motor de BD	PostgreSQL 15
Patrón de acceso	Repository Pattern
Microservicios con persistencia	ms-proyectos · ms-recursos · ms-analitica
Orquestación	Docker Compose con volúmenes nombrados

1. ¿Qué es Prisma ORM?

Prisma es un ORM (Object-Relational Mapper) de siguiente generación para Node.js y TypeScript. A diferencia de ORMs tradicionales como Sequelize o TypeORM, Prisma separa claramente la definición del esquema (schema.prisma), la generación del cliente tipado y las migraciones. Esto permite una capa de acceso a datos predecible, segura en tipos y fácilmente mantenible.

Ventajas aplicadas en Innovatech:

- **Seguridad de tipos:** el cliente generado refleja exactamente el esquema, eliminando errores de nombres de columnas.
- **Migraciones declarativas:** los cambios al esquema generan archivos SQL versionados automáticamente.
- **Consultas expresivas:** API fluida con include, where, orderBy sin necesidad de SQL manual.
- **Aislamiento de servicios:** cada microservicio tiene su propia base de datos y cliente Prisma, garantizando independencia.

2. Modelo de datos por microservicio

2.1 ms-proyectos

```
model Proyecto {
  id Int @id @default(autoincrement())
  nombre String
  descripcion String?
  estado Estado @default(EN_PROCESO)
  fechaInicio DateTime @default(now())
  fechaFin DateTime?
  tareas Tarea[] // Relación 1:N
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

```

}

model Tarea {
  id Int @id @default(autoincrement())
  titulo String
  descripcion String?
  estado Estado @default(EN_PROCESO)
  responsableId Int? // Referencia externa a ms-recursos
  proyectoId Int
  proyecto Proyecto @relation(fields: [proyectoId], references: [id])
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

enum Estado { PENDIENTE EN_PROCESO COMPLETADO CANCELADO }

```

La relación entre Proyecto y Tarea es **uno-a-muchos (1:N)**: un proyecto puede tener múltiples tareas. La clave foránea *proyectoId* en Tarea referencia el ID del proyecto padre. El campo *responsableId* no tiene una FK real hacia ms-recursos (cada microservicio gestiona su propia BD), siguiendo el patrón de **base de datos por servicio**.

2.2 ms-recursos

```

model Empleado {
  id Int @id @default(autoincrement())
  nombre String
  email String @unique
  cargo String
  habilidades String[] // Array nativo de PostgreSQL
  disponible Boolean @default(true)
  horasSemana Int @default(40)
  asignaciones Asignacion[] // Relación 1:N
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}

model Asignacion {
  id Int @id @default(autoincrement())
  empleadoId Int
  empleado Empleado @relation(fields: [empleadoId], references: [id])
  proyectoId Int // Referencia lógica al ms-proyectos
  horasAsig Int
  fechaInicio DateTime @default(now())
  fechaFin DateTime?
  createdAt DateTime @default(now())
}

```

2.3 ms-analitica

```

model KpiSnapshot {
  id Int @id @default(autoincrement())
  fecha DateTime @default(now())
  totalProyectos Int
  proyectosActivos Int
  tareasCompletadas Int
  tasaUtilizacion Float // % empleados no disponibles
  createdAt DateTime @default(now())
}

```

ms-analitica almacena **snapshots históricos** de KPIs calculados consultando a ms-proyectos y ms-recursos en tiempo real mediante llamadas HTTP (axios). Cada solicitud a GET /kpis guarda un registro en KpiSnapshot, permitiendo analizar tendencias históricas.

3. Repository Pattern en ms-proyectos

El microservicio ms-proyectos implementa el **Repository Pattern**, separando la lógica de acceso a datos de la capa de rutas HTTP. Esto mejora la **testeabilidad** (permite mockear el repositorio en tests) y la **mantenibilidad** (cambiar de ORM no afecta las rutas).

```
// repository.js
const proyectoRepository = {
  findAll: () => prisma.proyecto.findMany({ include: { tareas: true } }),
  findById: (id) => prisma.proyecto.findUnique({ where: { id: Number(id) },
    include: { tareas: true } }),
  create: (data) => prisma.proyecto.create({ data }),
  update: (id, data) => prisma.proyecto.update({ where: { id: Number(id) }, data }),
  delete: (id) => prisma.proyecto.delete({ where: { id: Number(id) } }),
}
```

4. Configuración de conexión y Docker

Cada microservicio lee la variable de entorno **DATABASE_URL** desde su archivo .env. En Docker Compose, los servicios de PostgreSQL se definen con volúmenes persistentes para que los datos sobrevivan reinicios del contenedor.

```
# docker-compose.yml (fragmento)
services:
  db-proyectos:
    image: postgres:15-alpine
    environment:
      POSTGRES_DB: innovatech_proyectos
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres123
    volumes:
      - db_proyectos_data:/var/lib/postgresql/data # Volumen persistente

  ms-proyectos:
    build: ./services/ms-proyectos
    environment:
      DATABASE_URL: postgresql://postgres:postgres123@db-proyectos:5432/innovatech_proyectos
    depends_on:
      - db-proyectos

volumes:
  db_proyectos_data: # Declaración del volumen nombrado
  db_recursos_data:
  db_analitica_data:
```

5. Resumen de modelos y relaciones

Servicio	Modelos	Relaciones	BD
ms-proyectos	Proyecto, Tarea	1 Proyecto → N Tareas	innovatech_proyectos
ms-recursos	Empleado, Asignacion	1 Empleado → N Asignaciones	innovatech_recursos

ms-analitica	KpiSnapshot	(Ninguna — datos calculados)	innovatech_analitica
--------------	-------------	------------------------------	----------------------

Nota sobre integridad referencial entre servicios: siguiendo el patrón "Database per Service", no existen claves foráneas entre bases de datos distintas. La consistencia eventual se garantiza a nivel de aplicación: ms-analitica consulta a los otros servicios vía HTTP, y ms-proyectos almacena responsableId como referencia lógica sin FK real hacia ms-recursos.